



23CSE203

DATA STRUCTURES AND ALGORITHM

Submitted by	
Name	B.SAI HASHINI
Roll.no	AV.SC.U4CSE25101
Section	CSE-B

SUBMITTED TO	DR. RAJ KUMAR BATCHU
DESIGNATION	ASST.PROFFESSOR (Sl. Grade)
DEPARTMENT	SCHOOL OF COMPUTING

Credit	5
--------	---

Index

S.no	Title	Page.no	Remark
1.	A) A rocket launch system displays a countdown before launch. Instead of using loop, implement the countdown using recursion. Write a recursive program to display a countdown from n to 1 followed by launch. B) Bank calculates the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate p^n (power), where P=principal growth factor, n=no.of years. C) Write a recursive program to print the first n terms of fibonacci series D) An hr department stores employee ids in a list develop a recursive program to search for a given employee id. Write a recursive function to search for an employee id in the list. E) In a logistics the no.of possible ways to arrange n different parcels calculate using the factorial (n!). develop a recursive program to calculate the factorial of given number.	4-13	
2.	A) write a python program to perform linear search on the given input B) Write a python program to perform binary search on the given	14-18	

	input. Verify by taking sorted array and unsorted array as a list.		
3.	A) write a python program to implement bubble sort, where the elements are taken from user. B) write a python program to implement selection sort, where the elements are taken from user. C) write a python program to implement insertion sort, where the elements are taken from user.	19-23	
4.	A) write a python program to implement merge sort, where the elements are taken from user. B) write a python program to implement quick sort, where the elements are taken from user.	23-29	
5.	A)Write a python program to implement a single linked list with the below operations: A.create linked list B.insert at beginning C.Insert at end D.Insert at index E.Delete by value F.Delete first node G.Delete last node H.Count number of nodes I.Display/traverse J.exit B)Write a python program to implement a double linked list with the below operations: A.create linked list B.insert at beginning C.Insert at end D.Insert at index E.Delete by value	30-60	

	F.Delete first node G.Delete last node H.Count number of nodes I.Display/traverse J.exit C) Write a python program to implement a circular linked list with the below operations: A.create linked list B.insert at beginning C.Insert at end D.Insert at index E.Delete by value F.Delete first node G.Delete last node H.Count number of nodes I.Display/traverse J.exit		
6.			
7.			

Week-1

A) Aim: A rocket launch system displays a countdown before launch. Instead of using loop, implement the countdown using recursion. Write a recursive program to display a countdown from n to 1 followed by launch.

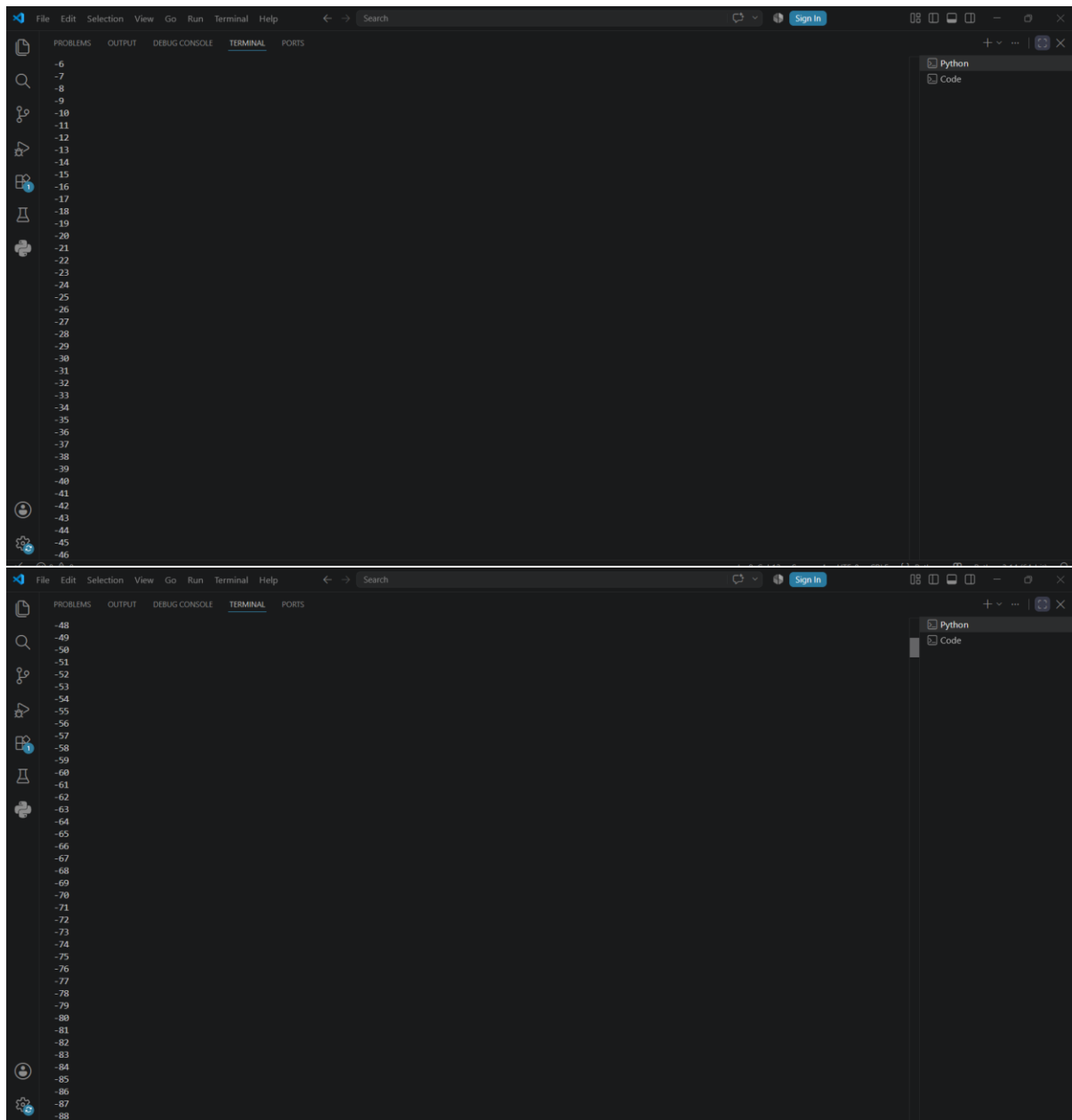
Program:

```
#av.sc.u4cse25101
def countdown(n):
    if n == 0:
        print("launch")
    else:
        print(n)
        countdown(n-1)

countdown(5)
```

Output:

```
PS C:\Users\SAI HASHINI\OneDrive\Desktop\textbooks\DSA Lab\week 1>
ktop/textbooks/DSA Lab/week 1/LAB1.A.py"
5
4
3
2
1
launch
```



Explanation:

- It takes a number as input from the user and stores it in num.
- It creates a function named launch().
- It starts the countdown from 10 by assigning $i = 10$.
- It prints the value of i as long as $\text{num} \leq i$.
- It decreases the value of i by 1 after each iteration.

- When the loop ends, it prints "Rocket launched" and executes the function using launch().
- For negative input, the function keeps decreasing instead of reaching 0, causing infinite recursion and eventually a Recursion Error.

Error table:

s.no	Error obtained	Rectification
1.	Infinite loop	Iteration at correct place
2.	Calling function at wrong place	Placed with correct syntax

B) Aim: Bank calculates the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate p^n (power), where P =principal growth factor, n =no. of years.

Program:

```

C: > Users > SAI HASHINI > OneDrive > Desktop > textbooks > DSA Lab > week 1 > lab1.B.py > ..
1  #av.sc.u4cse25101
2  def power(p, n):
3      if n == 0:
4          return 1
5      elif n < 0:
6          return 1 / power(p, -n)
7      else:
8          return p * power(p, n - 1)
9
10 principal = 1000
11 rate = 0.05
12 years = -3
13
14 amount = principal * power(1 + rate, years)
15
16 print("Final amount after", years, "years is:", amount)
17

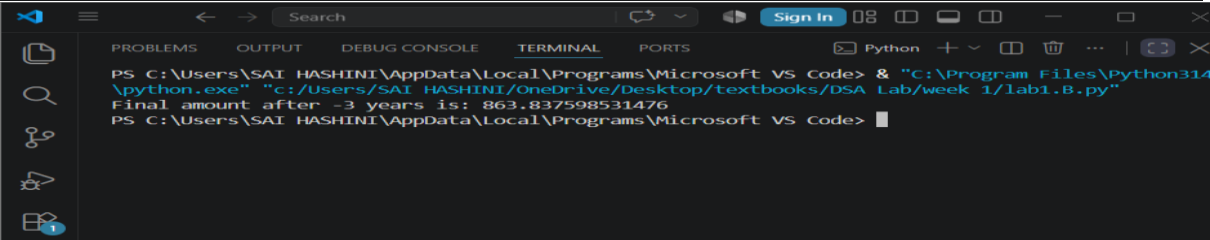
```

Output:

```

PS C:\Users\SAI HASHINI\OneDrive\Desktop\textbooks\DSA Lab\week 1>
Final amount after 3 years is: 1157.6250000000002
PS C:\Users\SAI HASHINI\OneDrive\Desktop\textbooks\DSA Lab\week 1>

```



Explanation:

- The program takes the principal growth factor p and number of years n as input.
- It defines a recursive function $\text{power}(p, n)$.
- If n is 0, the function returns 1 (base case).
- Otherwise, it returns $p * \text{power}(p, n-1)$.
- The recursive calls continue until n becomes 0.
- The final value of p^n is displayed as the output.

- For a negative exponent, calculate the reciprocal instead of continuing recursion downward indefinitely.
- Error table:

s.no	Error obtained	Rectification
1.	Return 0	Return 1
2.	print	return

C) Aim: Write a recursive program to print the first n terms of fibanocci series

Program:

```
#av.SC.04CSE25101
def fibonacci(n):
    if n < 0:
        return "Number of terms cannot be negative"
    if n == 0:
        return 0
    elif n == 1:
        return 1
    else:
        return fibonacci(n - 1) + fibonacci(n - 2)

n = int(input("Enter number of terms: "))

for i in range(n):
    print(fibonacci(i), end=" ")
```

Output:

```
Enter number of terms: 5
0 1 1 2 3
PS C:\Users\SAI HASHINI\OneDrive\Desktop\textbooks\DSA Lab\week 1>
```

```
Enter number of terms: -3
PS C:\Users\SAI HASHINI\AppData\Local\Programs\Microsoft VS Code> 
```

Explanation:

- For entering the number as input from user we defined as n
- We declared variable for formula.
- If input number is less than one then it return “1”
- Or it will goes under fibanocci iteration.
- If x is not present in the list, recursion reaches the end and returns False.

Error:

s.no	Error obtained	Rectification
1.	Forgot to declare $n \leq 1$	rectified
2.	Formula mistake	rectified

D) Aim: An hr department stores employee ids in a list develop a recursive program to search for a given employee id. Write a recursive function to search for an employee id in the list.

Program:

```
#av.sc.u4cse25101
def search(a, x, i=0):
    if i == len(a):
        return False
    if a[i] == x:
        return True
    return search(a, x, i + 1)

ids = [101, 205, 310, 415, 520]

print(search(ids, 310))
```

Output:

```
PS C:\Users\SAI HASHINI\OneDrive\Desktop\textbooks\DSA Lab\week 1> pyth
True
PS C:\Users\SAI HASHINI\OneDrive\Desktop\textbooks\DSA Lab\week 1> 310
310
PS C:\Users\SAI HASHINI\OneDrive\Desktop\textbooks\DSA Lab\week 1>
PS C:\Users\SAI HASHINI\AppData\Local\Programs\Microsoft VS Code> & "C:\Program Files\Python314\python.exe" "c:/Users/SAI HASHINI/OneDrive/Desktop/textbooks/DSA Lab/week 1/lab1.c.py"
False
```

Explanation:

- The program creates a function to search for an employee ID in a list.
- It checks one employee ID at a time, starting from the first ID.
- If the entered employee ID matches the current ID, it returns True.
- If it does not match, the function moves to the next ID using recursion.
- If all employee IDs are checked and no match is found, it returns False.

- Finally, the program displays "Employee ID found." if the ID exists; otherwise, it displays "Employee ID not found."
- Error:

s.no	Error obtained	Rectification
1.	Emp lst	Emp_list
2.	print	Return

E) Aim: In a logistics the no.of possible ways to arrange n different parcels calculate using the factorial (n!). develop a recursive program to calculate the factorial of given number.

Program:

```

1  #av.sc.u4cse25101
2  def factorial(n):
3      if n < 0:
4          return "not defined"
5      if n == 0 or n == 1:
6          return 1
7      else:
8          return n * factorial(n - 1)
9  num = int(input("Enter a number: "))
10 result = factorial(num)
11 print(f"The factorial of {num} is {result}")
12

```

Output:

```

Enter a number: 5
The factorial of 5 is 120
PS C:\Users\SAI HASHINI\OneDrive\Desktop\textbooks\DSA Lab\week 1>

```

```
PS C:\Users\SAI HASHINI\AppData\Local\Programs\Microsoft VS Code> & "C:\Program Files\Python\python.exe" "c:/Users/SAI HASHINI/OneDrive/Desktop/textbooks/DSA Lab/week 1/factorial.py"
Enter a number: -8
The factorial of -8 is not defined
PS C:\Users\SAI HASHINI\AppData\Local\Programs\Microsoft VS Code> █
```

Explanation:

- The program defines a recursive function `fact()` to calculate the factorial of a number.
- It takes a number as input from the user.
- If the number is 1, the function returns 1 (base case).
- Otherwise, it multiplies the number by the factorial of the previous number ($n-1$).
- The recursive calls continue until the number becomes 1.
- Finally, the program prints the factorial of the given number.
- Negative numbers do not have a factorial, so the program should handle them separately instead of recursing forever.

Error table:

s.no	Error obtained	Rectification
1.	$n+1$	$n-1$
2.	Syntax error	Rectified

WEEK-2

A) AIM: Write a python program to perform linear search on the given input.

Program:

```
def linear_search_sorted(arr,key):  
    for i in range(len(arr)):  
        if arr[i]==key:  
            return i  
        elif arr[i]> key:  
            return -1  
    return -1  
  
arr = [1,4,6,8,9,12,15,16,19]  
key = int(input("enter the element to be found:"))  
result=linear_search_sorted(arr,key)  
  
if result==-1:  
    print("element not found")  
else:  
    print(f"element found at index : {result} ")
```

Output:

```
enter the element to be found:12  
element found at index : 5  
PS C:\Users\SAI HASHINI\OneDrive\Desktop\textbooks\DSA Lab\week 2>
```

```
enter the element to be found:20
element not found
```

Explanation:

- In linear search we can append elements in sorted or unsorted way also.
- After placing the target we need to check the all the elements in a order.
- The time complicity is $O(n)$.
- The target element may be found at beginning or last or at middle.
- If the key is greater than every element in the sorted array, the search reaches the end and returns -1.

Error table:

s.no	Error obtained	Rectification
1.	Return placed in wrong place	Corrected later
2.	Calling the function	Rectified

B) Aim: Write a python program to perform binary search on the list.

Program: Unsorted

```
#av.sc.u4cse25101
def binary_search(arr, key):
    i, j = 0, len(arr) - 1
    while i <= j:
        mid = (i + j) // 2
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            i = mid + 1
        else:
            j = mid - 1
    return -1
arr = [15, 3, 9, 20, 7, 12, 18]
arr.sort()
print("Sorted array:", arr)
key = int(input("Enter the element to search: "))
result = binary_search(arr, key)
if result != -1:
    print(f"Element found at index {result} in the sorted array")
else:
    print(f"Element not found in the array")
```

Output:

```
Sorted array: [3, 7, 9, 12, 15, 18, 20]
Enter the element to search: 15
Element found at index 4 in the sorted array
PS C:\Users\SAI HASHINI\OneDrive\Desktop\textbooks\DSA Lab\week 2>
```

```
Sorted array: [3, 7, 9, 12, 15, 18, 20]
Enter the element to search: 25
Element not found in the array
PS C:\Users\SAI HASHINI>
```

Program: Sorted

```

#av.sc.u4cse25101|
def binary_search(arr, key):
    i, j = 0, len(arr) - 1
    while i <= j:
        mid = (i + j) // 2
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            i = mid + 1
        else:
            j = mid - 1
    return -1
arr = [3,7,9,12,15,18,20]
key = int(input("Enter the element to search: "))
result = binary_search(arr, key)
if result != -1:
    print(f"Element found at index {result} in the")
else:
    print(f"Element not found in the array")

```

Output:

```

Enter the element to search: 12
Element found at index 3 in the sorted array
PS C:\Users\SAI HASHINI\OneDrive\Desktop\textbooks\DSA Lab\week 2>

```

```

Enter the element to search: 2
Element not found in the array

```

Explanation:

- The binary search should always be executed in sorted order.
- Either it can be ascending or descending order.
- So the program is in unsorted order so we need to do sorting first later the binary search is executed.

- Error table:

s.no	Error obtained	Rectification
1.	Mistakes in main logic	Rectified
2.	Errors in conditions	Rectified

WEEK-3

A) AIM: write a python program to implement bubble sort, where the elements are taken from user.

Program:

```

1  def bubble_sort(arr):
2      n = len(arr)
3
4      for i in range(n - 1):
5          for j in range(n - i - 1):
6              if arr[j] > arr[j + 1]:
7                  arr[j], arr[j + 1] = arr[j + 1], arr[j]
8
9
10 n = int(input("Enter no. of elements: "))
11
12 if n <= 0:
13     print("Number of elements must be positive")
14 else:
15     arr = []
16
17     print("Enter elements:")
18     for i in range(n):
19         arr.append(int(input()))
20
21     bubble_sort(arr)
22
23     print("Sorted Array:")
24     for element in arr:
25         print(element, end=" ")

```

Output:

```

Enter no. of elements: 5
Enter elements:
12
25
46
7
97
Sorted Array:
7 12 25 46 97
PS C:\Users\SAI HASHINI\OneDrive\Desktop\textbooks\DSA Lab\week 3>

```

```

Enter no. of elements: 0
Number of elements must be positive
PS C:\Users\SAI HASHINI\AppData\Local\Programs\Microsoft VS Code>

```

Explanation:

- Bubble sort is a sorting technique used to arrange elements in **ascending or descending order**.
- The program may contain elements in **unsorted order**, so adjacent elements are compared and swapped if they are in the wrong order.
- This process is repeated for multiple passes until the **entire array is sorted**.
- For $n = 0$, the program now prints the error message without trying to use an undefined arr.

Error table:

s.no	Error obtained	Rectification
1.	Mistakes in main logic	Rectified
2.	Errors in conditions	Rectified

B) AIM: write a python program to implement insertion sort, where the elements are taken from user.

Program:

```
#AV.SC.04CSE25101
def insertion_sort(arr):
    n = len(arr)
    for i in range(1, n):
        key = arr[i]
        j = i - 1

        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j = j - 1

        arr[j + 1] = key
    return arr
n = int(input("Enter no. of elements: "))
arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))
insertion_sort(arr)
print("Sorted Array:")
for element in arr:
    print(element, end=" ")
```

Output:

```
Enter no. of elements: 0
Enter the elements:
Sorted Array:
PS C:\Users\SAI HASHINI\AppData\Local\Programs\Microsoft VS Code>
```

```
Enter no. of elements: 5
Enter the elements:
24
4
12
75
35
Sorted Array:
4 12 24 35 75
PS C:\Users\SAI HASHINI\OneDrive\Desktop\textbooks\DSA Lab\week 3>
```

Explanation:

- Insertion sort is a sorting technique used to arrange elements in **ascending or descending order**.
- The program takes one element at a time and **inserts it into its correct position** in the already sorted part.
- This process is repeated until **all elements are sorted**.
- When the number of elements is zero, the array is empty and insertion sort simply performs no iterations.

Error table:

s.no	Error obtained	Rectification
1.	Mistakes in main logic	Rectified
2.	Errors in conditions	Rectified

C) AIM: write a python program to implement selection sort, where the elements are taken from user.

Program:

```
#AV.SC.U4CSE25101
def selection_sort(arr):
    n = len(arr)

    for i in range(n - 1):
        min_index = i

        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j

        arr[i], arr[min_index] = arr[min_index], arr[i]

n = int(input("Enter no. of elements: "))
arr = []
print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))
print("Original Array:", arr)
selection_sort(arr)
print("Sorted Array:", arr)
```

Output:

```
Enter no. of elements: 5
Enter the elements:
24
76
35
12
9
Original Array: [24, 76, 35, 12, 9]
```

```
Enter no. of elements: 0
Enter the elements:
Original Array: []
Sorted Array: []
```

Explanation:

- Selection sort is a sorting technique used to arrange elements in **ascending or descending order**.
- The program finds the **smallest element** from the unsorted part and swaps it with the first element of that part.
- This process is repeated until the **entire array is sorted**.
- When $n = 0$, the array is empty and selection sort performs no iterations, so the program runs normally.

Error table:

s.no	Error obtained	Rectification
1.	Mistakes in main logic	Rectified
2.	Errors in conditions	Rectified

WEEK-3

A) AIM: write a python program to implement bubble sort, where the elements are taken from user.

Program:

```

1  def bubble_sort(arr):
2      n = len(arr)
3
4      for i in range(n - 1):
5          for j in range(n - i - 1):
6              if arr[j] > arr[j + 1]:
7                  arr[j], arr[j + 1] = arr[j + 1], arr[j]
8
9
10 n = int(input("Enter no. of elements: "))
11
12 if n <= 0:
13     print("Number of elements must be positive")
14 else:
15     arr = []
16
17     print("Enter elements:")
18     for i in range(n):
19         arr.append(int(input()))
20
21     bubble_sort(arr)
22
23     print("Sorted Array:")
24     for element in arr:
25         print(element, end=" ")

```

Output:

```

Enter no. of elements: 5
Enter elements:
12
25
46
7
97
Sorted Array:
7 12 25 46 97
PS C:\Users\SAI HASHINI\OneDrive\Desktop\textbooks\DSA Lab\week 3>

```

```

Enter no. of elements: 0
Number of elements must be positive
PS C:\Users\SAI HASHINI\AppData\Local\Programs\Microsoft VS Code>

```

Explanation:

- Bubble sort is a sorting technique used to arrange elements in **ascending or descending order**.
- The program may contain elements in **unsorted order**, so adjacent elements are compared and swapped if they are in the wrong order.
- This process is repeated for multiple passes until the **entire array is sorted**.
- For $n = 0$, the program now prints the error message without trying to use an undefined arr.

Error table:

s.no	Error obtained	Rectification
1.	Mistakes in main logic	Rectified
2.	Errors in conditions	Rectified

WEEK-4

A) AIM: write a python program to implement merge sort, where the elements are taken from user.

Program:

```

av.sc.u4cse25101
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        left = arr[:mid]
        right = arr[mid:]

        merge_sort(left)
        merge_sort(right)

        i = j = k = 0

        while i < len(left) and j < len(right):
            if left[i] < right[j]:
                arr[k] = left[i]
                i += 1
            else:
                arr[k] = right[j]
                j += 1
            k += 1

        while i < len(left):
            arr[k] = left[i]
            i += 1
            k += 1

        while j < len(right):
            arr[k] = right[j]
            j += 1
            k += 1

```

```

n = int(input("Enter number of elements: "))
arr = []

```

```

print("Enter elements:")

```

```

print("Enter elements:")
for i in range(n):
    arr.append(int(input()))

```

```

merge_sort(arr)

```

```

print("Sorted Array:")
for element in arr:
    print(element, end=" ")

```

Output:

```

PS C:\Users\SAT HASHINI\OneDrive\Desktop\text
Enter number of elements: 7
Enter elements:
10
25
9
16
7
26
14
Sorted Array:
7 9 10 14 16 25 26

```

Explanation:

- **Merge Sort** divides the array into two halves repeatedly until each part has one element.
- The smaller arrays are then **sorted recursively**.
- The **while** loop compares elements of the left and right arrays.
- The smaller element is placed into the original array in sorted order.
- Finally, the remaining elements are added, giving the **completely sorted array**.

Error table:

s.no	Error obtained	Rectification
1.	Mistakes in main logic	Rectified
2.	Errors in conditions	Rectified

B) AIM: write a python program to implement Quick sort, where the elements are taken from user.

Program:

```

AV.SC.U4CSE25101
def quick_sort(a, low, high):
    if low < high:
        i = low
        j = high
        pivot = low

        while i < j:
            while i < len(a) and a[i] <= a[pivot]:
                i += 1
            while a[j] > a[pivot]:
                j -= 1

            if i < j:
                a[i], a[j] = a[j], a[i]

            a[j], a[pivot] = a[pivot], a[j]

        quick_sort(a, low, j - 1)
        quick_sort(a, j + 1, high)

a = list(map(int, input("Enter numbers to sort: ").split()))

quick_sort(a, 0, len(a) - 1)

print("Sorted Array:")
print(a)

```

Output:

```

PS C:\Users\SAI HASHINI\OneDrive\Desktop\textbooks\DSA\w
Enter numbers to sort: 10 25 9 16 7 26 14
Sorted Array:
[7, 9, 10, 14, 16, 25, 26]

```

Explanation:

- **Quick Sort** selects an element as the **pivot** and divides the array around it.
- Elements smaller than the pivot are moved to the **left**, and larger elements to the **right**.
- The pivot is placed in its correct position.

- Quick Sort is then applied **recursively** to the left and right parts.
- This process continues until the entire array is **sorted**.

Error table:

s.no	Error obtained	Rectification
1.	Mistakes in main logic	Rectified
2.	Errors in conditions	Rectified

Week-5

A) Write a python program to implement a single linked list with the below operations:

- A. create linked list
- B. insert at beginning
- C. Insert at end
- D. Insert at index
- E. Delete by value
- F. Delete first node
- G. Delete last node
- H. Count number of nodes
- I. Display/traverse
- J. exit

Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class SLL:
    def __init__(self):
        self.head = None

    # 1. Create Linked List
    def create(self):
        n = int(input("Enter number of nodes: "))
```

```

    for i in range(n):
        data = int(input("Enter data: "))
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
        else:
            temp = self.head
            while temp.next is not None:
                temp = temp.next
            temp.next = new_node

    print("Linked List created successfully")

# 2. Insert at Beginning
def insert_beginning(self):
    data = int(input("Enter data: "))

    new_node = Node(data)

    new_node.next = self.head
    self.head = new_node
    print(f"Inserted {data} at the beginning.")

# 3. Insert at End
def insert_end(self):
    data = int(input("Enter data: "))

    new_node = Node(data)

    if self.head is None:
        self.head = new_node
        return

    temp = self.head

    while temp.next is not None:
        temp = temp.next

    temp.next = new_node
    print(f"Inserted {data} at the end.")

# 4. Insert at Index
def insert_at_index(self):
    data = int(input("Enter data: "))
    index = int(input("Enter index: "))

```

```

    if index < 0:
        print("Invalid index")
        return

    if index == 0:
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node
        return

    temp = self.head

    for i in range(index - 1):
        if temp is None:
            print("Index out of range")
            return
        temp = temp.next

    if temp is None:
        print("Index out of range")
        return

    new_node = Node(data)

    new_node.next = temp.next
    temp.next = new_node
    print(f"Inserted {data} at index {index}.")

# 5. Delete from Beginning
def delete_beginning(self):

    if self.head is None:
        print("List is empty")
        return

    self.head = self.head.next
    print(f"Deleted beginning node.")

# 6. Delete from End
def delete_end(self):

    if self.head is None:
        print("List is empty")
        return

```

```

        if self.head.next is None:
            self.head = None
            return

        temp = self.head

        while temp.next.next is not None:
            temp = temp.next

        temp.next = None
        print(f"Deleted last node.")

# 7. Delete from Index
def delete_at_index(self):

    index = int(input("Enter index: "))

    if self.head is None:
        print("List is empty")
        return

    if index < 0:
        print("Invalid index")
        return

    if index == 0:
        self.head = self.head.next
        return

    temp = self.head

    for i in range(index - 1):
        if temp.next is None:
            print("Index out of range")
            return
        temp = temp.next

    if temp.next is None:
        print("Index out of range")
        return

    temp.next = temp.next.next
    print(f"Deleted the node value.")

# 8. Count Nodes
def count_nodes(self):

```

```

        count = 0
        temp = self.head

        while temp is not None:
            count += 1
            temp = temp.next

        print("Number of nodes:", count)
# 9. Display
def display(self):

    if self.head is None:
        print("List is empty")
        return

    temp = self.head

    while temp is not None:
        print(temp.data, end=" -> ")
        temp = temp.next

    print("None")

# Main Program

sll = SLL()

while True:

    print("\n----- SINGLY LINKED LIST -----")
    print("1. Create Linked List")
    print("2. Insert at Beginning")
    print("3. Insert at End")
    print("4. Insert at Index")
    print("5. Delete from Beginning")
    print("6. Delete from End")
    print("7. Delete from Index")
    print("8. Count Number of nodes")
    print("9. Display")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

```

```
if choice == 1:
    sll.create()

elif choice == 2:
    sll.insert_beginning()

elif choice == 3:
    sll.insert_end()

elif choice == 4:
    sll.insert_at_index()

elif choice == 5:
    sll.delete_beginning()

elif choice == 6:
    sll.delete_end()

elif choice == 7:
    sll.delete_at_index()

elif choice == 8:
    sll.count_nodes()

elif choice == 9:
    sll.display()

elif choice == 10:
    print("Exiting from the program...")
    break

else:
    print("Invalid choice")
```

Output:

```

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 1
Enter number of nodes: 4
Enter data: 12
Enter data: 24
Enter data: 56
Enter data: 78
Linked List created successfully

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 9
12 -> 24 -> 56 -> 78 -> None

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 2
Enter data: 5

```

```

Enter data: 5
Inserted 5 at the beginning.

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 9
5 -> 12 -> 24 -> 56 -> 78 -> None

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 3
Enter data: 89
Inserted 89 at the end.

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 9
5 -> 12 -> 24 -> 56 -> 78 -> 89 -> None

```

```

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 5
Deleted beginning node.

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 9
12 -> 24 -> 56 -> 78 -> 89 -> None

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 6
Deleted last node.

```

```

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 7
Enter index: 3
Deleted the node value.

```

```

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 8
Number of nodes: 3

```

```

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 9
12 -> 24 -> 56 -> None

```

```

----- SINGLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 10
Exiting from the program...
PS C:\Users\SAI HASHINI\OneDrive\Desktop\textbooks\DSA\week 5>

```

Explanation:

- Node represents each node and stores **data** and the address of the **next node**.
- SLL maintains the linked list using `self.head`, which points to the first node.
- `create()` creates the linked list by adding nodes one by one.
- `insert` operations add a new node at the **beginning, end, or specified index**.
- `delete` operations remove a node from the **beginning, end, or specified index**.
- `count_nodes()` counts the total number of nodes, while `display()` prints all node values.
- The **main program** uses a menu-driven loop to perform the selected linked-list operation

Error table:

s.no	Error obtained	Rectification
1.	def _init_(self, data):	def init (self, data):
2.	def insert_beginning():	def insert_beginning(self):
3.	self.head = data	new_node = Node(data) self.head = new_node

Time complexity table:

Operation	Best Case	Average Case	Worst Case	Space Complexity
Create	O(1)	O(1)	O(1)	O(n)
Insert at Beginning	O(1)	O(1)	O(1)	O(1)
Insert at End	O(1) (if tail pointer maintained)	O(n)	O(n)	O(1)
Insert at Position	O(1) (if head/tail known)	O(n)	O(n)	O(1)
Delete from Beginning	O(1)	O(1)	O(1)	O(1)
Delete from End	O(n)	O(n)	O(n)	O(1)
Delete by Value	O(1) (if at head)	O(n)	O(n)	O(1)
Search	O(1) (if at head)	O(n)	O(n)	O(1)
Count Nodes	O(n)	O(n)	O(n)	O(1)
Display	O(n)	O(n)	O(n)	O(1)

B) Write a python program to implement a double linked list with the below operations:

- A. create linked list
- B. insert at beginning
- C. Insert at end
- D. Insert at index
- E. Delete by value
- F. Delete first node
- G. Delete last node
- H. Count number of nodes
- I. Display/traverse
- J. exit

Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None

class DLL:
    def __init__(self):
        self.head = None

    # 1. Create Linked List
    def create(self):
        n = int(input("Enter number of nodes: "))

        for i in range(n):
            data = int(input("Enter data: "))
```

```

        new_node = Node(data)

        if self.head is None:
            self.head = new_node
        else:
            temp = self.head

            while temp.next is not None:
                temp = temp.next

            temp.next = new_node
            new_node.prev = temp

    print("Linked List created successfully")

# 2. Insert at Beginning
def insert_beginning(self):
    data = int(input("Enter data: "))

    new_node = Node(data)

    new_node.next = self.head

    if self.head is not None:
        self.head.prev = new_node

    self.head = new_node

    print(f"Inserted {data} at the beginning.")

# 3. Insert at End
def insert_end(self):
    data = int(input("Enter data: "))

    new_node = Node(data)

    if self.head is None:
        self.head = new_node
        print(f"Inserted {data} at the end.")
        return

    temp = self.head

    while temp.next is not None:
        temp = temp.next

```

```

temp.next = new_node
new_node.prev = temp

print(f"Inserted {data} at the end.")

# 4. Insert at Index
def insert_at_index(self):
    data = int(input("Enter data: "))
    index = int(input("Enter index: "))

    if index < 0:
        print("Invalid index")
        return

    if index == 0:
        new_node = Node(data)

        new_node.next = self.head

        if self.head is not None:
            self.head.prev = new_node

        self.head = new_node

    print(f"Inserted {data} at index {index}.")
    return

temp = self.head

for i in range(index - 1):
    if temp is None:
        print("Index out of range")
        return

    temp = temp.next

if temp is None:
    print("Index out of range")
    return

new_node = Node(data)

new_node.next = temp.next
new_node.prev = temp

if temp.next is not None:

```

```

        temp.next.prev = new_node

    temp.next = new_node

    print(f"Inserted {data} at index {index}.")

# 5. Delete from Beginning
def delete_beginning(self):

    if self.head is None:
        print("List is empty")
        return

    self.head = self.head.next

    if self.head is not None:
        self.head.prev = None

    print("Deleted beginning node.")

# 6. Delete from End
def delete_end(self):

    if self.head is None:
        print("List is empty")
        return

    if self.head.next is None:
        self.head = None
        print("Deleted last node.")
        return

    temp = self.head

    while temp.next is not None:
        temp = temp.next

    temp.prev.next = None

    print("Deleted last node.")

# 7. Delete from Index
def delete_at_index(self):

    index = int(input("Enter index: "))

```

```

    if self.head is None:
        print("List is empty")
        return

    if index < 0:
        print("Invalid index")
        return

    if index == 0:
        self.head = self.head.next

        if self.head is not None:
            self.head.prev = None

        print("Deleted the node value.")
        return

    temp = self.head

    for i in range(index):
        if temp is None:
            print("Index out of range")
            return

        temp = temp.next

    if temp is None:
        print("Index out of range")
        return

    if temp.next is not None:
        temp.next.prev = temp.prev

    temp.prev.next = temp.next

    print("Deleted the node value.")

# 8. Count Nodes
def count_nodes(self):

    count = 0
    temp = self.head

    while temp is not None:
        count += 1
        temp = temp.next

```

```

        print("Number of nodes:", count)

# 9. Display
def display(self):

    if self.head is None:
        print("List is empty")
        return

    temp = self.head

    while temp is not None:
        print(temp.data, end=" <-> ")
        temp = temp.next

    print("None")

# Main Program

dll = DLL()

while True:

    print("\n----- DOUBLY LINKED LIST -----")
    print("1. Create Linked List")
    print("2. Insert at Beginning")
    print("3. Insert at End")
    print("4. Insert at Index")
    print("5. Delete from Beginning")
    print("6. Delete from End")
    print("7. Delete from Index")
    print("8. Count Number of nodes")
    print("9. Display")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        dll.create()

    elif choice == 2:
        dll.insert_beginning()

    elif choice == 3:

```

```
        dll.insert_end()

    elif choice == 4:
        dll.insert_at_index()

    elif choice == 5:
        dll.delete_beginning()

    elif choice == 6:
        dll.delete_end()

    elif choice == 7:
        dll.delete_at_index()

    elif choice == 8:
        dll.count_nodes()

    elif choice == 9:
        dll.display()

    elif choice == 10:
        print("Exiting from the program...")
        break

    else:
        print("Invalid choice")
```

```

----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 1
Enter number of nodes: 4
Enter data: 23
Enter data: 45
Enter data: 36
Enter data: 78
Linked List created successfully

----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 2
Enter data: 11
Inserted 11 at the beginning.

----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 9
11 <-> 23 <-> 45 <-> 36 <-> 78 <-> None

```

```

----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 3
Enter data: 98
Inserted 98 at the end.

----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 9
11 <-> 23 <-> 45 <-> 36 <-> 78 <-> 98 <-> None

----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 4
Enter data: 4
Enter index: 55
Index out of range

```

```

----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 9
11 <-> 23 <-> 45 <-> 36 <-> 78 <-> 98 <-> None

```

```

----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 5
Deleted beginning node.

```

```

----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 6
Deleted last node.

```

```

----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 7
Enter index: 4
Index out of range

```

```

----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 8
Number of nodes: 4

```

```

----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 9
23 <-> 45 <-> 36 <-> 78 <-> None

```

```

----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 10
Exiting from the program...
PS C:\Users\SAI HASHINI\OneDrive\Desktop\textbooks\DSA\week 6> 

```

Explanation:

- Node represents each node and stores **data**, the address of the **previous node**, and the **next node**.
- DLL maintains the linked list using `self.head`, which points to the first node.

- `create()` creates the doubly linked list by connecting nodes using both `prev` and `next`.
- `insert` operations add a new node at the **beginning, end, or specified index**.
- `delete` operations remove a node from the **beginning, end, or specified index** while updating both links.
- `count_nodes()` counts the total nodes, while `display()` prints the nodes in forward direction.
- The **main program** uses a menu-driven loop to perform the selected doubly linked-list operation.

Error table:

s.no	Error obtained	Rectification
1.	<code>self.next = None</code>	<code>self.prev = None</code> <code>self.next = None</code>
2.	<code>while temp is not None:</code> <code>print(temp.data)</code>	<code>while temp is not None:</code> <code>print(temp.data)</code> <code>temp = temp.next</code>

Time complexity table:

Operation	Best Case	Average Case	Worst Case	Space Complexity
Create	$O(1)$	$O(1)$	$O(1)$	$O(n)$
Insert at Beginning	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Insert at End	$O(1)$ (if tail pointer maintained)	$O(n)$	$O(n)$	$O(1)$
Insert at Position	$O(1)$ (if head/tail known)	$O(n)$	$O(n)$	$O(1)$

Delete from Beginning	O(1)	O(1)	O(1)	O(1)
Delete from End	O(1) (if tail pointer maintained)	O(n)	O(n)	O(1)
Delete by Value	O(1) (if at head/tail)	O(n)	O(n)	O(1)
Search	O(1) (if at head/tail)	O(n)	O(n)	O(1)
Traverse Forward	O(n)	O(n)	O(n)	O(1)
Traverse Backward	O(n)	O(n)	O(n)	O(1)

C) Write a python program to implement a double linked list with the below operations:

- A. create linked list
- B. insert at beginning
- C. Insert at end
- D. Insert at index
- E. Delete by value
- F. Delete first node
- G. Delete last node
- H. Count number of nodes
- I. Display/traverse
- J. exit

Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

```

class CLL:
    def __init__(self):
        self.head = None

    # 1. Create Linked List
    def create(self):
        n = int(input("Enter number of nodes: "))

        for i in range(n):
            data = int(input("Enter data: "))
            new_node = Node(data)

            if self.head is None:
                self.head = new_node
                new_node.next = self.head
            else:
                temp = self.head

                while temp.next != self.head:
                    temp = temp.next

                temp.next = new_node
                new_node.next = self.head

        print("Linked List created successfully")

    # 2. Insert at Beginning
    def insert_beginning(self):
        data = int(input("Enter data: "))
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            new_node.next = self.head
        else:
            temp = self.head

            while temp.next != self.head:
                temp = temp.next

            new_node.next = self.head
            temp.next = new_node
            self.head = new_node

```

```

        print(f"Inserted {data} at the beginning.")

# 3. Insert at End
def insert_end(self):
    data = int(input("Enter data: "))
    new_node = Node(data)

    if self.head is None:
        self.head = new_node
        new_node.next = self.head
        print(f"Inserted {data} at the end.")
        return

    temp = self.head

    while temp.next != self.head:
        temp = temp.next

    temp.next = new_node
    new_node.next = self.head

    print(f"Inserted {data} at the end.")

# 4. Insert at Index
def insert_at_index(self):
    data = int(input("Enter data: "))
    index = int(input("Enter index: "))

    if index < 0:
        print("Invalid index")
        return

    if index == 0:
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
            new_node.next = self.head
        else:
            temp = self.head

            while temp.next != self.head:
                temp = temp.next

            new_node.next = self.head
            temp.next = new_node

```

```

        self.head = new_node

        print(f"Inserted {data} at index {index}.")
        return

    if self.head is None:
        print("Index out of range")
        return

    temp = self.head

    for i in range(index - 1):
        temp = temp.next

    if temp == self.head:
        print("Index out of range")
        return

    new_node = Node(data)

    new_node.next = temp.next
    temp.next = new_node

    print(f"Inserted {data} at index {index}.")

# 5. Delete from Beginning
def delete_beginning(self):

    if self.head is None:
        print("List is empty")
        return

    if self.head.next == self.head:
        self.head = None
        print("Deleted beginning node.")
        return

    temp = self.head

    while temp.next != self.head:
        temp = temp.next

    self.head = self.head.next
    temp.next = self.head

    print("Deleted beginning node.")

```

```

# 6. Delete from End
def delete_end(self):

    if self.head is None:
        print("List is empty")
        return

    if self.head.next == self.head:
        self.head = None
        print("Deleted last node.")
        return

    temp = self.head

    while temp.next.next != self.head:
        temp = temp.next

    temp.next = self.head

    print("Deleted last node.")

# 7. Delete from Index
def delete_at_index(self):

    index = int(input("Enter index: "))

    if self.head is None:
        print("List is empty")
        return

    if index < 0:
        print("Invalid index")
        return

    if index == 0:
        if self.head.next == self.head:
            self.head = None
        else:
            temp = self.head

            while temp.next != self.head:
                temp = temp.next

            self.head = self.head.next
            temp.next = self.head

```

```

        print("Deleted the node value.")
        return

    temp = self.head

    for i in range(index - 1):
        temp = temp.next

    if temp == self.head:
        print("Index out of range")
        return

    if temp.next == self.head:
        print("Index out of range")
        return

    temp.next = temp.next.next

    print("Deleted the node value.")

# 8. Count Nodes
def count_nodes(self):

    if self.head is None:
        print("Number of nodes:", 0)
        return

    count = 0
    temp = self.head

    while True:
        count += 1
        temp = temp.next

        if temp == self.head:
            break

    print("Number of nodes:", count)

# 9. Display
def display(self):

    if self.head is None:
        print("List is empty")
        return

```

```

        temp = self.head

        while True:
            print(temp.data, end=" -> ")
            temp = temp.next

            if temp == self.head:
                break

        print("Head")

# Main Program

c11 = CLL()

while True:

    print("\n----- CIRCULAR LINKED LIST -----")
    print("1. Create Linked List")
    print("2. Insert at Beginning")
    print("3. Insert at End")
    print("4. Insert at Index")
    print("5. Delete from Beginning")
    print("6. Delete from End")
    print("7. Delete from Index")
    print("8. Count Number of nodes")
    print("9. Display")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        c11.create()

    elif choice == 2:
        c11.insert_beginning()

    elif choice == 3:
        c11.insert_end()

    elif choice == 4:
        c11.insert_at_index()

    elif choice == 5:

```

```
        cll.delete_beginning()

    elif choice == 6:
        cll.delete_end()

    elif choice == 7:
        cll.delete_at_index()

    elif choice == 8:
        cll.count_nodes()

    elif choice == 9:
        cll.display()

    elif choice == 10:
        print("Exiting from the program...")
        break

    else:
        print("Invalid choice")
```

```

----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 1
Enter number of nodes: 4
Enter data: 23
Enter data: 45
Enter data: 36
Enter data: 78
Linked List created successfully

----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 2
Enter data: 11
Inserted 11 at the beginning.

----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 9
11 <-> 23 <-> 45 <-> 36 <-> 78 <-> None

```

```

----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 3
Enter data: 98
Inserted 98 at the end.

----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 9
11 <-> 23 <-> 45 <-> 36 <-> 78 <-> 98 <-> None

----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 4
Enter data: 4
Enter index: 55
Index out of range

```

```

----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 9
11 <-> 23 <-> 45 <-> 36 <-> 78 <-> 98 <-> None

```

```

----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 5
Deleted beginning node.

```

```

----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 6
Deleted last node.

```

```

----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 7
Enter index: 4
Index out of range

```

```

----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 8
Number of nodes: 4

```

```

----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 9
23 <-> 45 <-> 36 <-> 78 <-> None

```

```

----- DOUBLY LINKED LIST -----
1. Create Linked List
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete from Beginning
6. Delete from End
7. Delete from Index
8. Count Number of nodes
9. Display
10. Exit
Enter your choice: 10
Exiting from the program...
PS C:\Users\SAI HASHINI\OneDrive\Desktop\textbooks\DSA\week 6> 

```

Explanation:

- The Node class stores data and a next pointer, while CLL manages the circular linked list.
- In a CLL, the last node always points back to the head instead of None.

- `create()` creates the list, while `insert_beginning()`, `insert_end()`, and `insert_at_index()` add nodes.
- `delete_beginning()`, `delete_end()`, and `delete_at_index()` remove nodes from different positions.
- `count_nodes()` counts all nodes by traversing until it reaches the head again.
- `display()` prints the elements and stops when traversal returns to the head.
- The main program provides a menu to perform all operations until the user chooses Exit.

Error table:

s.no	Error obtained	Rectification
1.	while temp is not None: temp = temp.next #checking for None	while temp.next != self.head: temp = temp.next
2.	temp.next = new_node #forgetting circular connection	temp.next = new_node new_node.next = self.head
3.	self.head = self.head.next #deleting first node without updating last node	self.head = self.head.next temp.next = self.head

TIME COMPLEXITY TABLE:

Operation	Best Case	Average Case	Worst Case	Space Complexity
-----------	-----------	--------------	------------	------------------

Create	$O(1)$	$O(n^2)$	$O(n^2)$	$O(n)$
Insert at Beginning	$O(n)$	$O(n)$	$O(n)$	$O(1)$
Insert at End	$O(n)$	$O(n)$	$O(n)$	$O(1)$
Insert at Index	$O(1)$	$O(n)$	$O(n)$	$O(1)$
Delete from Beginning	$O(n)$	$O(n)$	$O(n)$	$O(1)$
Delete from End	$O(n)$	$O(n)$	$O(n)$	$O(1)$
Delete from Index	$O(1)$	$O(n)$	$O(n)$	$O(1)$
Count Nodes	$O(n)$	$O(n)$	$O(n)$	$O(1)$
Display	$O(n)$	$O(n)$	$O(n)$	$O(1)$